Flight Controller per Razzo a Stabilizzazione Attiva

Active Fin Stabilization — GNC System

Microcontrollore:	Teensy (alta prestazione)
IMU:	Adafruit BNO055 (9 assi, Sensor Fusion)
Altimetro:	Adafruit BMP390 (barometro aerospaziale)
Telemetria:	DX-LR03 Modulo LoRa
Attuatori:	5 servomotori (4 alette + paracadute)
Versione FW:	v1.0

Indice

1 Panoramica del Progetto **3**

1.1 Obiettivo . . . 3

1.2 Filosofia del Sistema . . . 3

2 Hardware: Componenti e Pinout **4**

2.1 Il Cervello: Scheda Teensy . . . 4

2.2 Sottosistema di Navigazione Inerziale . . . 4

2.2.1 IMU — Adafruit BNO055 (Giroscopio / Sensore di Assetto) . . . 4

2.2.2 Altimetro — Adafruit BMP390 (Barometro di Precisione) . . . 5

2.3 Sottosistema di Comunicazione — Modulo LoRa . . . 5

2.4 Sottosistema di Attuazione — Servomotori . . . 6

2.4.1 4 Servi di Controllo (Alette Aerodinamiche) . . . 6

2.4.2 Servo del Paracadute (Meccanismo di Espulsione) . . . 6

3 Architettura Elettrica e Schema di Cablaggio (Wiring Topology) **7**

3.1 Regole Fondamentali di Cablaggio . . . 7

3.2 Tabella di Cablaggio Completa (Master Wiring List) . . . 8

4 Costanti di Default e Limiti Meccanici **10**

4.1 Parametri dell'Algoritmo di Controllo (PID) . . . 10

4.2 Limiti Cinematici dei Servomotori . . . 11

4.3 Guadagni del Filtro Inerziale (Kalman) . . . 11

4.4 Specifiche di Trasmissione e Temporizzazione . . . 11

5 Setup: Sequenza di Inizializzazione **13**

5.1 Configurazione dei Bus di Comunicazione . . . 13

5.2 Validazione Sensori e Logica Fail-Safe . . . 13

5.3 Armamento Attuatori e Trim Aerodinamico . . . 14

6 Il Loop: Algoritmi di Volo e Navigazione Inerziale **15**

6.1 Interblocco di Calibrazione Remota (Remote Pad Calibration) . . . 15

6.1.1 La Soluzione: Calibrazione Comandata e Feedback Visivo . . . 15

6.2 Temporizzazione e Calcolo del Delta Time (Δt) 16
6.2.1 Implementazione Firmware 16
6.3 Acquisizione Inerziale e Filtraggio EMA 17
6.3.1 Il Problema: Spettro Vibrazionale 17
6.3.2 La Soluzione: Filtro IIR del Primo Ordine 17
6.3.3 Implementazione Firmware 17
6.4 Matrice di Trasformazione (DCM) e Correzione Vettoriale 3D 17
6.4.1 Il Problema: Body Frame vs. Inertial Frame 17
6.4.2 La Soluzione: Proiezione Vettoriale 18
6.4.3 Implementazione Firmware 18
6.5 Passo 4 — Acquisizione Barometrica e Sensor Fusion (Filtro di Kalman) 18
6.5.1 Acquisizione Barometrica e Traslazione QFE 19
6.5.2 Il Filtro di Kalman (Variante Estesa Alpha-Beta) 19
6.5.3 Output del Sistema Inerziale 20
6.6 Passo 5 — Interfaccia Cablata GCS (Umbilical Link) 20
6.6.1 Dizionario dei Comandi Seriali (ICD - Interface Control Document) 20
6.7 Passo 6 — Telemetria via LoRa 23
6.8 Passo 7 — Logica dell'Apogeo e Espulsione Paracadute 24

7 Miscelatore Aerodinamico e Attuazione Cinematica 25
7.1 Panoramica: Algoritmo di Control Allocation 25
7.2 Disaccoppiamento degli Assi e Logica Differenziale 25
7.2.1 Autorità di Beccheggio (Pitch) e Rollio 25
7.2.2 Autorità di Imbardata (Yaw) e Rollio 26
7.2.3 Autorità di Rollio (Controllo Collettivo) 26
7.3 Gestione della Saturazione e Clamping Elettronico 26
7.3.1 Modalità di Guasto per Saturazione Non Gestita 26
7.3.2 La Soluzione Matematica: Electronic End-Point Adjustment (EPA) 27

A Glossario dei Termini Tecnici 28

Capitolo 1

Panoramica del Progetto

1.1 Obiettivo

Questo progetto consiste nello sviluppo di un **computer di bordo (Flight Controller)** avanzato per un razzo amatoriale. Il sistema è in grado di:

- **Mantenere il razzo perfettamente verticale** durante la fase di spinta, compensando vento e asimmetrie del motore muovendo 4 alette aerodinamiche indipendenti;
- **Calcolare matematicamente l'apogeo** per espellere il paracadute in totale sicurezza, rilevando autonomamente il punto più alto della traiettoria;
- **Trasmettere la telemetria a terra in tempo reale** via radio LoRa, permettendo il monitoraggio completo del volo dalla stazione di terra (GCS).

1.2 Filosofia del Sistema

Il *cervello* del sistema è un microcontrollore ad altissime prestazioni della famiglia Teensy, indispensabile per gestire calcoli in virgola mobile e matrici trigonometriche in tempo reale. Il sistema GNC (**Guidance, Navigation and Control**) è strutturato in tre livelli funzionali:

1. **Navigation:** acquisizione e fusione dati dai sensori (IMU + barometro) con Filtro di Kalman;
2. **Guidance:** algoritmo PID a tre assi indipendenti per calcolare le correzioni necessarie;
3. **Control:** miscelatore aerodinamico (Mixer) che traduce le correzioni PID in deflessioni angolari fisiche dei servi.

Capitolo 2

Hardware: Componenti e Pinout

2.1 Il Cervello: Scheda Teensy

Ruolo nel Sistema

La Teensy è il **computer di bordo**. Rispetto a un normale Arduino, possiede una potenza di calcolo e una velocità nettamente superiori. Questo è vitale perché deve eseguire calcoli matematici complessi (l'algoritmo PID e il Filtro di Kalman) centinaia di volte al secondo senza mai bloccarsi.

2.2 Sottosistema di Navigazione Inerziale

Questi due moduli forniscono al computer di volo la **consapevolezza spaziale assoluta**. Condividono lo stesso **bus I2C**, un protocollo di comunicazione digitale che permette a più sensori di comunicare con la Teensy usando solo due cavi (più l'alimentazione).

2.2.1 IMU — Adafruit BNO055 (Giroscopio / Sensore di Assetto)

Ruolo nel Software

È l'**organo dell'equilibrio** del razzo: capisce come il razzo è inclinato nello spazio in ogni istante del volo.

Perché questo modulo: a differenza dei sensori base, il BNO055 possiede un *co-processore interno dedicato alla Sensor Fusion*. Invece di far calcolare matrici complesse alla Teensy, il sensore le consegna già pronte e pulite. È un sensore intelligente a **9 assi**.

Dati estratti:

- **Orientamento Spaziale (Angoli di Eulero):** fornisce l'inclinazione esatta in gradi su tre assi:

  - **Pitch** (Beccheggio): inclinazione avanti/dietro;
  - **Roll** (Rollio): rotazione assiale;
  - **Yaw** (Imbardata): rotazione destra/sinistra.

  Questi angoli vengono dati direttamente in pasto all'algoritmo PID.

- **Accelerazione Lineare:** fornisce la spinta netta del motore (depurata matematicamente dalla gravità terrestre), dato vitale per calcolare l'accelerazione verticale reale.

2.2.2 Altimetro — Adafruit BMP390 (Barometro di Precisione)

Ruolo nel Software

È l'**altimetro di precisione**: sente i minimi cambiamenti della pressione atmosferica per calcolare la quota del razzo dal suolo.

Perché questo modulo: il BMP390 è uno dei barometri più precisi sul mercato, progettato in ambito aerospaziale per rilevare micro-variazioni della pressione atmosferica. Durante il volo, l'aria diventa meno densa man mano che il razzo sale.

Dati estratti: altitudine grezza. Questo dato, pur essendo soggetto al rumore aerodinamico del vento, viene unito all'accelerazione del BNO055 tramite il **Filtro di Kalman** per ottenere una traccia millimetrica della quota e, soprattutto, della velocità verticale necessaria per rilevare l'apogeo.

2.3 Sottosistema di Comunicazione — Modulo LoRa

Ruolo nel Software

È il **ponte radio** con la Stazione di Terra (GCS — Ground Control Station). Durante tutto il volo, invia alla stazione di terra i dati di volo in tempo reale.

Perché LoRa: la tecnologia *Long Range* garantisce trasmissioni a lunghissima distanza (chilometri) con una larghezza di banda bassa ma estremamente robusta contro le interferenze — perfetta per inviare la stringa CSV con tutti i dati di volo a **5 Hz** (un pacchetto ogni 200 ms).

Dati trasmessi (pacchetto CSV): Tempo, Quota, Velocità, Apogeo, Pitch, Roll, Yaw, Accelerazioni 3D, Uscite PID e Stato Servi (oltre 15 campi per pacchetto).

2.4 Sottosistema di Attuazione — Servomotori

Il razzo utilizza un'aerodinamica **cruciforme** (quattro alette disposte a croce) più un meccanismo di espulsione. Tutti i componenti ricevono un segnale **PWM** (Pulse Width Modulation) dalla Teensy per posizionarsi all'angolo esatto calcolato dal miscelatore aerodinamico.

2.4.1 4 Servi di Controllo (Alette Aerodinamiche)

- **Aletta Sinistra** (Pin 6): lavora in coppia differenziale con la Destra per controllare l'Imbardata (Yaw); lavora all'unisono con tutte le altre per controllare il Rollio (Roll).
- **Aletta Destra** (Pin 7): speculare alla Sinistra.
- **Aletta Frontale** (Pin 8): lavora in coppia differenziale con la Posteriore per controllare il Beccheggio (Pitch).
- **Aletta Posteriore** (Pin 9): speculare alla Frontale.

2.4.2 Servo del Paracadute (Meccanismo di Espulsione)

Funzionamento Binario

A differenza degli altri servi, il servo del paracadute **non compie micro-correzioni**. Funziona come un interruttore meccanico:

- **CHIUSO** (0°): carica una molla o uno sportello — rimane fisso in questa posizione durante l'intera fase di salita;
- **APERTO** (90°): scatta istantaneamente all'apogeo per liberare la calotta di recupero.

Capitolo 3

Architettura Elettrica e Schema di Cablaggio (Wiring Topology)

Questo capitolo definisce le specifiche fisiche per l'assemblaggio dell'hardware (Hardware Integration). Per garantire la totale replicabilità del vettore spaziale, la mappatura non si limita al *routing* dei segnali logici, ma definisce rigorosamente la topologia di alimentazione per prevenire interferenze elettromagnetiche (EMI) e cadute di tensione (Brownout).

3.1 Regole Fondamentali di Cablaggio

Topologia ad Alimentazione Separata (Power Isolation)

I cinque servomotori sotto carico aerodinamico generano picchi di assorbimento (Stall Current) che possono superare i 3-5 Ampere complessivi. **I servomotori non devono MAI essere alimentati dal pin 5V o 3.3V della Teensy.**
È imperativo utilizzare un **circuito UBEC (Universal Battery Eliminator Circuit)** dedicato per fornire 5V/6V esclusivamente ai cavi rossi (VCC) dei servi. La Teensy fornirà unicamente il segnale PWM. **Tutti i Ground (GND) del sistema — Batteria, UBEC, Teensy e Sensori — devono essere uniti in un unico nodo equipotenziale (Common Ground).**

3.2 Tabella di Cablaggio Completa (Master Wiring List)

La seguente matrice descrive ogni singola connessione punto-a-punto necessaria per l'assemblaggio del Flight Controller.

Terminale Modulo	Terminale Teensy	Note Ingegneristiche
Sottosistema di Navigazione (Bus I2C in Daisy-Chain)		
BNO055 (VIN)	Teensy Pin `3.3V`	Alimentazione logica sensore IMU
BNO055 (GND)	Nodo Comune `GND`	Riferimento di massa
BNO055 (SDA)	Teensy Pin `18` (SDA0)	Linea Dati (In parallelo col Barometro)
BNO055 (SCL)	Teensy Pin `19` (SCL0)	Linea Clock (In parallelo col Barometro)
BMP390 (VIN)	Teensy Pin `3.3V`	Alimentazione logica sensore Altimetro
BMP390 (GND)	Nodo Comune `GND`	Riferimento di massa
BMP390 (SDA)	Teensy Pin `18` (SDA0)	Linea Dati (In parallelo con IMU)
BMP390 (SCL)	Teensy Pin `19` (SCL0)	Linea Clock (In parallelo con IMU)
Sottosistema di Telemetria (Bus UART Hardware)		
LoRa (VIN / VCC)	Teensy Pin `3.3V`	Alimentazione logica transceiver RF
LoRa (GND)	Nodo Comune `GND`	Riferimento di massa
LoRa (TX)	Teensy Pin `17` (RX4)	**Crossover:** Trasmissione LoRa → Ricezione Teensy
LoRa (RX)	Teensy Pin `16` (TX4)	**Crossover:** Trasmissione Teensy → Ricezione LoRa
Interfaccia Utente (LED di Stato FSM)		
LED Rosso (Anodo)	Teensy Pin `2`	Segnale di Stato ARMATO (Richiede resistenza in serie, es. 220 Ω)
LED Rosso (Catodo)	Nodo Comune `GND`	Chiusura circuito a massa
LED Verde (Anodo)	Teensy Pin `3`	Segnale di Stato DISARMATO (Richiede resistenza in serie)
LED Verde (Catodo)	Nodo Comune `GND`	Chiusura circuito a massa
Sottosistema Attuatori (Bus PWM e Potenza di Rete)		
Tutti i Servi (Rosso)	**Output 5V dell'UBEC**	Potenza meccanica
Tutti i Servi (Nero)	Nodo Comune `GND`	Chiusura circuito logico
Servo Sinistra (Segnale)	Teensy Pin `6`	Controllo differenziale Yaw + Rollio
Servo Destra (Segnale)	Teensy Pin `7`	Controllo differenziale Yaw + Rollio
Servo Frontale (Segnale)	Teensy Pin `8`	Controllo differenziale Pitch + Rollio
Servo Posteriore (Segnale)	Teensy Pin `9`	Controllo differenziale Pitch + Rollio
Servo Paracadute (Segnale)	Teensy Pin `10`	Trigger binario evento Apogeo (0°/90°)

Capitolo 4

Costanti di Default e Limiti Meccanici

Questo capitolo definisce i parametri operativi di base e i vincoli cinematici del Flight Controller. I valori qui riportati costituiscono la *baseline* nominale per il primo collaudo e sono stati pre-calcolati per garantire stabilità statica e dinamica. Al fine di ottimizzare i tempi di sviluppo, l'architettura software consente la ricalibrazione dinamica di tali costanti direttamente sulla rampa di lancio tramite l'interfaccia cablata (GCS), senza necessità di ricompilare il firmware.

4.1 Parametri dell'Algoritmo di Controllo (PID)

L'algoritmo Proporzionale-Integrale-Derivativo (PID) regola la reazione degli attuatori in base all'errore di assetto. I guadagni nominali sono stati impostati per favorire una risposta reattiva alle perturbazioni (vento al traverso) mantenendo un margine di fase sufficiente a prevenire oscillazioni distruttive.

Tabella 4.1: Costanti PID di default per il controllo d'assetto

Parametro di Controllo	Simbolo	Valore Nominale
Guadagno Proporzionale (Reattività)	K_p	1.5
Guadagno Integrale (Compensazione statica)	K_i	0.1
Guadagno Derivativo (Smorzamento)	K_d	0.5
Saturazione Integratore	`LIMITE_INTEGRATORE`	$\pm$ 100.0

Gestione del Windup Integrale

Il termine integrale è fisicamente limitato dalla costante `LIMITE_INTEGRATORE`. Questa saturazione software, nota come meccanismo di *Anti-Windup*, è critica durante lo stazionamento sulla rampa e nei primissimi istanti di volo. Previene l'accumulo di un errore fittizio che porterebbe il controllore a comandare correzioni

violente e fuori scala (saturando prematuramente i servi) non appena il vettore abbandona il rail di guida.

4.2 Limiti Cinematici dei Servomotori

Per prevenire lo stallo aerodinamico delle alette (fin stall) e il sovraccarico elettrico degli attuatori in caso di richieste di controllo estreme, le corse dei servomotori sono severamente vincolate via software tramite la funzione di *clamping*.

Tabella 4.2: Vincoli angolari e saturazioni meccaniche degli attuatori

Condizione Cinematica	Costante Firmware	Angolo
Trim neutro aerodinamico	`CENTRO`	90°
Saturazione cinematica inferiore	`LIMITE_SERVO_MIN`	45°
Saturazione cinematica superiore	`LIMITE_SERVO_MAX`	135°
Stato di volo (Paracadute armato)	`ANGOLO_CHIUSO`	0°
Evento Apogeo (Paracadute espulso)	`ANGOLO_APERTO`	90°

4.3 Guadagni del Filtro Inerziale (Kalman)

Il Filtro di Kalman (nella sua variante Alpha-Beta semplificata) esegue la fusione dei dati (sensor fusion) tra l'accelerazione lineare ad alta frequenza e la lettura barometrica a bassa frequenza. I coefficienti scelti riflettono la necessità di fidarsi maggiormente della cinematica predetta dall'IMU, utilizzando il barometro principalmente per l'abbattimento della deriva stocastica (drift) sul lungo periodo.

Tabella 4.3: Coefficienti di innovazione del Filtro di Kalman

Variabile di Stato	Costante	Peso / Innovazione
Aggiornamento Altitudine (Correzione Barometrica)	K_{alt}	0.15
Aggiornamento Velocità (Inerzia Accelerometro)	K_{vel}	0.05

4.4 Specifiche di Trasmissione e Temporizzazione

L'architettura di comunicazione è divisa tra il monitoraggio ad alta velocità sulla rampa e la telemetria a lungo raggio durante il volo. Il temporizzatore del *main loop* garantisce la coerenza temporale delle equazioni differenziali del PID.

- **Frequenza di campionamento e controllo:** 50 Hz (Loop rigorosamente limitato a $\Delta t \geq 20$ ms);
- **Frequenza di downlink telemetrico:** 5 Hz (1 pacchetto dati completo ogni 200 ms);
- **Baud rate Umbilical Link (USB Serial):** 115.200 baud (High-speed debug pre-lancio);
- **Baud rate Telemetria RF (LoRa UART):** 9.600 baud (Ottimizzato per range estremi e minima corruzione pacchetti);
- **Clock Bus I2C (Sensori):** 400 kHz (Fast Mode, minimizza i tempi di blocco del processore durante le letture IMU).

Capitolo 5

Setup: Sequenza di Inizializzazione

La routine di `setup()` costituisce il *Power-On Self-Test* del Flight Controller. Questa fase esegue una rigorosa checklist aerospaziale in modalità puramente sequenziale e bloccante. L'obiettivo è garantire che ogni sottosistema (comunicazione, sensoristica e attuazione) sia in uno stato nominale prima di cedere il controllo alla Macchina a Stati del *main loop*.

5.1 Configurazione dei Bus di Comunicazione

L'architettura hardware richiede flussi di dati paralleli a latenza controllata. La configurazione iniziale prevede:

1. **Umbilical Link (Serial USB):** Inizializzata a 115.200 baud, fornisce un canale ad alta velocità per il debug e il *Live Tuning* dei parametri del PID sulla rampa di lancio.
2. **Downlink Telemetrico (Serial4):** Inizializzata a 9.600 baud per interfacciarsi con il trasmettitore LoRa, massimizzando il rapporto segnale/rumore (SNR) per le trasmissioni a lungo raggio.
3. **Bus I2C (Wire):** Il clock primario viene forzato a 400 kHz (*Fast Mode*). Questa forzatura è critica per minimizzare i tempi di blocco del processore durante il campionamento (polling) dell'IMU e dell'altimetro nel ciclo di volo.

5.2 Validazione Sensori e Logica Fail-Safe

L'integrità dei dati inerziali e barometrici è un requisito assoluto per la stabilità del vettore. L'inizializzazione non si limita ad accendere i sensori, ma ne configura i registri hardware interni:

- **BNO055 (IMU):** Viene forzato l'utilizzo del cristallo oscillatore esterno (`setExtCrystalUse`) per garantire la massima precisione termica delle letture angolari.

- **BMP390 (Barometro):** Viene configurato con un *Oversampling* 8x per la temperatura e 32x per la pressione, affiancati da un Filtro IIR (Infinite Impulse Response) di coefficiente 3. Questo pre-filtraggio hardware elimina il rumore indotto dalle turbolenze d'aria all'interno del vano avionico.

Logica di Retry Sequenziale

A causa dei possibili ritardi di accensione (*power-up delay*) dei componenti sul bus I2C, il software implementa un ciclo `while` di diagnostica che tenta l'handshake con i sensori per un massimo di 3 tentativi (`MAX_TENTATIVI`).

Interblocco Hardware Critico

Qualora un sensore non risponda entro i tentativi previsti, il sistema dichiara un'avaria di livello critico (Fatal Error). Il microcontrollore entra in un *infinite loop* bloccante (looping trap) segnalato da un lampeggio di errore del LED. Questo meccanismo *Fail-Safe* inibisce fisicamente il proseguimento del codice, rendendo impossibile un lancio "alla cieca".

5.3 Armamento Attuatori e Trim Aerodinamico

Come ultimo passaggio di sicurezza, i pin PWM vengono attivati e collegati ai servomotori:

- Le quattro superfici di controllo (alette) vengono comandate in hard-lock alla posizione di **CENTRO (90°)**. Ciò imposta il trim aerodinamico in posizione neutrale, prevenendo asimmetrie di attrito (*drag*) durante lo scorrimento sulla rampa.
- Il servomotore di espulsione viene forzato e mantenuto attivamente alla costante `ANGOLO_SERVO_PARACADUTE_CHIUSO`, vincolando il carico di recupero.

La conclusione nominale del `setup()` abilita i LED di stato e cede il flusso di esecuzione al Flight Controller attivo.

Capitolo 6

Il Loop: Algoritmi di Volo e Navigazione Inerziale

Il `loop()` principale opera come un sistema di controllo *real-time* ad alta frequenza. La rigorosa temporizzazione (Fixed Time Step) è un requisito sistemistico fondamentale per garantire la stabilità numerica del calcolo differenziale e integrale all'interno del controllore PID.

6.1 Interblocco di Calibrazione Remota (Remote Pad Calibration)

Se la calibrazione ambientale venisse eseguita all'interno del `setup()` durante l'istante di accensione (Power-On), il sistema registrerebbe un assetto falso. L'operatore, infatti, deve ancora manipolare il vettore per chiudere il vano avionico, trasportarlo e infilarlo sulla rampa di lancio (Rail). Una calibrazione prematura porterebbe il PID a riconoscere come "verticale" un angolo casuale, causando un testacoda catastrofico al momento del decollo.

6.1.1 La Soluzione: Calibrazione Comandata e Feedback Visivo

Per ovviare a questo problema, il firmware implementa un'architettura di **Remote Pad Calibration**. All'accensione, il `loop()` principale è congelato da un interblocco logico. Il razzo non esegue calcoli e si limita ad ascoltare la telemetria seriale. Solo quando il vettore è perfettamente verticale sulla rampa , è possibile inviare il comando di calibrazione (`K`), innescando la subroutine `eseguiCalibrazionePad()`.

Questa subroutine esegue la calibrazione dei sensori accomagnati da **Feedback Visivo (HMI)**:

1. **Fase di Stabilizzazione (2 secondi):** Il sistema attende che i filtri hardware dei sensori si riempiano. Il LED verde esegue un lampeggio di avviso a media frequenza (100 ms ON / 100 ms OFF).
2. **Fase di Campionamento Barometrico (1 secondo):** Il barometro acquisisce 20 campioni (distanziati da 50 ms) per stabilire la quota del suolo (QFE). Durante questo ciclo, i LED lampeggiano ad altissima frequenza (50 ms ON / 50 ms OFF), confermando l'elaborazione dei dati in corso.
3. **Acquisizione Zero Inerziale:** L'IMU scatta un'istantanea degli Angoli di Eulero attuali, impostandoli come *setpoint* (0, 0, 0) per i tre anelli PID.
4. **Fase di conferma e sblocco:** Entrambi i led si accendono per una frazione di secondo per poi rispegnersi avvisando la completa riuscita della calibrazione dei moduli. La variabile `sensori_calibrati` viene commutata a `true`, sbloccando la Macchina a Stati.

Prevenzione del D-Kick Temporale

L'ultima istruzione della calibrazione (`tempoPassato_loop = millis()`) è un accorgimento sistemistico vitale. Poiché la calibrazione impegna il processore per circa 3 secondi, omettere questa riga causerebbe l'iniezione di un Δt di 3.0 secondi nel primo ciclo del PID, innescando una reazione derivativa esplosiva (*Derivative Kick*). L'azzeramento forzato del cronometro assicura invece una transizione fluida all'algoritmo di volo.

6.2 Temporizzazione e Calcolo del Delta Time (Δt)

Una volta aver calibrato i sensori il firmware vincola la frequenza massima di esecuzione del blocco di controllo a **50 Hz** (periodo di 20 ms). Questa scelta bilancia la necessità di reattività aerodinamica con i limiti di banda del bus I2C.

6.2.1 Implementazione Firmware

L'algoritmo di temporizzazione esegue tre operazioni di validazione:

1. **Attesa Attiva (Polling):** Tramite la misurazione in millisecondi (`millis()`), il sistema valuta il tempo trascorso dal ciclo precedente (`tempoPassato_loop`). Se inferiore a 20 ms, il ciclo viene abortito tramite `return`.
2. **Risoluzione del Δt:** Al superamento della soglia, il tempo esatto viene convertito in secondi continui nella variabile `dt_loop`, parametro cruciale per le matrici di Kalman e i guadagni PID.

3. **Filtro Anti-Spike (Watchdog):** Qualora il sistema subisca un blocco anomalo (es. ritardo I/O) tale per cui `dt_loop > 0.5` secondi, il calcolo viene **scartato**. Questo interblocco previene l'iniezione di un Δt abnorme nell'equazione derivativa del PID, che altrimenti genererebbe un impulso di correzione distruttivo (D-Kick) sugli attuatori.

6.3 Acquisizione Inerziale e Filtraggio EMA

6.3.1 Il Problema: Spettro Vibrazionale

Durante la fase propulsiva, i motori a razzo a propellente solido generano un severo ambiente vibrazionale, caratterizzato da transienti di combustione e risonanze strutturali. La lettura grezza dell'accelerometro lungo l'asse longitudinale di spinta (a_x) risulta affetta da fluttuazioni stocastiche ad alta frequenza. L'integrazione numerica diretta di tale segnale introdurrebbe un errore cumulativo (drift) letale per la stima dell'apogeo.

6.3.2 La Soluzione: Filtro IIR del Primo Ordine

Per mitigare il rumore senza introdurre ritardi di fase (*lag*) inaccettabili, l'accelerazione viene processata tramite un **Filtro a Media Mobile Esponenziale (EMA)**. L'equazione alle differenze implementata è:

$$a_{x_filtrata}(n) = 0.8 \cdot a_{x_filtrata}(n-1) + 0.2 \cdot a_x(n)$$

6.3.3 Implementazione Firmware

Il microcontrollore interroga il co-processore del BNO055 richiedendo due vettori hardware pre-fusi: `VECTOR_EULER` (per gli angoli di assetto) e `VECTOR_LINEARACCEL` (per l'accelerazione netta, già priva del vettore gravitazionale). Immediatamente dopo, la variabile `accel_x` viene filtrata sovrascrivendo `accel_x_filtrata`. Il coefficiente 0.8 fornisce l'inerzia necessaria a smorzare i transienti, mentre il peso 0.2 garantisce il tracciamento della reale curva di spinta.

6.4 Matrice di Trasformazione (DCM) e Correzione Vettoriale 3D

6.4.1 Il Problema: Body Frame vs. Inertial Frame

Il sensore inerziale rileva le forze rispetto al **Body Frame** (il sistema di coordinate triassiale solidale al razzo). Per scopi di navigazione, è imperativo tradurre queste

accelerazioni nell'**Inertial Frame** (il sistema di coordinate fisso rispetto al suolo). In presenza di perturbazioni di assetto (es. inclinazione causata da vento al traverso), l'utilizzo della sola accelerazione assiale a_x sovrastimerebbe il reale guadagno di quota, poiché parte dell'energia propulsiva verrebbe dispersa nel moto traslazionale orizzontale.

6.4.2 La Soluzione: Proiezione Vettoriale

Il software compensa questo disallineamento geometrico eseguendo una trasformazione vettoriale tridimensionale. L'algoritmo applica una semplificazione della Matrice dei Coseni Direttori (DCM):

$$a_{verticale} = a_x \cos(\text{Pitch}) \cos(\text{Roll}) + a_y \sin(\text{Roll}) + a_z \sin(\text{Pitch})$$

La dinamica si scompone in due contributi:

- **Proiezione Assiale:** Il termine $a_x \cos(\text{Pitch}) \cos(\text{Roll})$ decurta matematicamente la frazione di spinta che non punta direttamente allo zenith.
- **Recupero Trasversale:** I termini in seno valutano le accelerazioni trasversali (a_y, a_z) che, a causa dell'inclinazione del vettore, proiettano una componente di forza risultante lungo l'asse Z dell'Inertial Frame.

6.4.3 Implementazione Firmware

Poiché le librerie matematiche standard del C/C++ (`math.h`) richiedono argomenti in radianti, il codice esegue primariamente una conversione degli angoli di Eulero tramite il fattore di conversione `(PI / 180.0)`. Successivamente, processa la somma pesata delle tre componenti accelerometriche. L'output, assegnato alla variabile `accel_verticale_vera`, fornisce l'accelerazione netta ed esatta contro la gravità terrestre, depurata da qualsiasi artefatto dovuto all'orientamento spaziale del vettore.

6.5 Passo 4 — Acquisizione Barometrica e Sensor Fusion (Filtro di Kalman)

La determinazione deterministica dell'istante di apogeo richiede la conoscenza esatta della velocità verticale (V_z). Il calcolo della velocità tramite pura derivazione numerica dell'altitudine barometrica risulterebbe catastrofico a causa del rumore aerodinamico (fluttuazioni di pressione indotte dall'effetto Bernoulli sui fori di scarico statico). Al contrario, la pura integrazione dell'accelerazione inerziale genererebbe una deriva (drift) incontrollabile. La soluzione implementata è un algoritmo di *Sensor Fusion* basato sulle equazioni predittore-correttore.

6.5.1 Acquisizione Barometrica e Traslazione QFE

Il sensore BMP390 acquisisce la pressione statica esterna. L'istruzione `readAltitude(1013.25)` utilizza la formula ipso-metrica standard basata sul QNH (pressione a livello del mare) per restituire una quota grezza (`global_altitudine_baro_raw`). Per le logiche di volo, tuttavia, è necessaria l'altitudine AGL (*Above Ground Level*). Il firmware esegue quindi una traslazione del sistema di riferimento sottraendo la tara pre-calcolata sulla rampa (`global_altitudineDiPartenza`), ottenendo `global_altitudineDalSuolo`.

6.5.2 Il Filtro di Kalman (Variante Estesa Alpha-Beta)

La funzione `aggiornaKalman()` implementa un osservatore di stato ad alta efficienza. L'algoritmo si divide rigorosamente in due fasi: **Predizione** (aggiornamento temporale basato sulla fisica) e **Correzione** (aggiornamento delle misure basato sui sensori).

Fase 1: Predizione Cinematica (State Extrapolation)

Il sistema utilizza l'accelerazione netta proiettata (a_z, ovvero `accel_verticale`) per propagare in avanti lo stato del razzo assumendo un moto uniformemente accelerato nel brevissimo intervallo Δt.

Le equazioni di predizione dello stato (altitudine $\hat{h}$ e velocità $\hat{v}$) sono:

$$\hat{h}_{k|k-1} = \hat{h}_{k-1} + \hat{v}_{k-1}\Delta t + \frac{1}{2}a_z\Delta t^2$$

$$\hat{v}_{k|k-1} = \hat{v}_{k-1} + a_z\Delta t$$

Nell'implementazione firmware, queste equazioni Newtoniane corrispondono esattamente alle variabili `predizione_alt` e `predizione_vel`. In questa frazione di millisecondo, il computer *stima* dove dovrebbe trovarsi il razzo basandosi esclusivamente sulla spinta del motore letta dall'IMU.

Fase 2: Correzione Statistica (Measurement Update)

Il sistema confronta ora l'altitudine predetta matematicamente con l'altitudine reale appena misurata dal barometro. La differenza tra questi due valori genera il segnale di **Innovazione** o **Errore Residuo**:

$$e_k = h_{baro} - \hat{h}_{k|k-1}$$

L'innovazione (`errore` nel codice) viene moltiplicata per i Guadagni di Kalman (K_{alt} e K_{vel}) per aggiornare le stime finali. I guadagni agiscono come coefficienti di "fiducia":

- `stima_altitudine = predizione_alt + (K_alt * errore);`
- `stima_velocita = predizione_vel + (K_vel * errore);`

6.5.3 Output del Sistema Inerziale

Il risultato di questa cascata matematica fornisce le due variabili di stato globali fondamentali per il *Flight Controller*:

- `stima_altitudine`: Una curva di quota fluida, reattiva e immune ai picchi di pressione del vento laterale.
- `stima_velocita`: La velocità verticale istantanea reale. È il differenziale esatto che permetterà alla Macchina a Stati (FSM) di rilevare l'inversione cinematica ($V_z < 0$) e dichiarare l'apogeo con precisione assoluta.

6.6 Passo 5 — Interfaccia Cablata GCS (Umbilical Link)

Funzione

Nelle fasi antecedenti il decollo, il sistema implementa una subroutine di comunicazione seriale asincrona via cavo, che funge da **cordone ombelicale** tra il Flight Controller e la GCS. Questo evita di saturare il canale radio LoRa durante le operazioni a terra.

6.6.1 Dizionario dei Comandi Seriali (ICD - Interface Control Document)

Il parser implementato nella funzione `leggiComandiDaTerra()` analizza il flusso di byte in ingresso sulla porta USB. L'algoritmo decodifica un set di istruzioni ASCII a singolo carattere (case-insensitive per i parametri di calibrazione), seguite opzionalmente da un *payload* numerico (parsing dei float o degli interi).

Di seguito è riportata la specifica formale del protocollo di comunicazione per le operazioni di pre-lancio (*On-Pad Operations*).

1. Comandi di Gestione dello Stato (Safety & Interlock)

Questi comandi commutano lo stato della Macchina a Stati Finiti (FSM), abilitando o inibendo l'attuazione aerodinamica.

1. Comandi di Inizializzazione e Gestione dello Stato (Safety & Interlock)

Questi comandi governano l'allineamento inerziale del sistema e commutano lo stato della Macchina a Stati Finiti (FSM), abilitando o inibendo l'attuazione aerodinamica.

- **Comando `K` (Calibrazione Pad):** Innesca la subroutine `eseguiCalibrazionePad()`. Il sistema stabilizza i sensori, elabora la media barometrica per il QFE (Zero del suolo) e acquisisce l'istantanea degli Angoli di Eulero (Zero Inerziale). Al termine, commuta la variabile `sensori_calibrati = true`.
  Nota di Sicurezza: Il comando è interbloccato. Se inviato mentre il sistema è in stato ARMED, la richiesta viene respinta per prevenire la sovrascrittura accidentale del *Reference Frame* durante il volo.
  Risultato a monitor: `[GCS] Inizio Calibrazione... NON TOCCARE IL RAZZO!` seguito dalla conferma dei dati acquisiti.

- **Comando `A` (ARM - Armamento):** Commuta il flag `razzo_armato_per_volo = true`. Rimuove il sezionatore logico di sicurezza.
  Interblocco Sequenziale: La richiesta viene categoricamente respinta se prima non è stata eseguita e validata la procedura di calibrazione (Comando `K`). Se autorizzato, il Flight Controller cede piena autorità al PID. Da questo momento, il vettore è considerato *Live*.
  Risultato a monitor: `[GCS] ATTENZIONE: SISTEMA ARMATO! PRONTO AL LANCIO!` (Oppure: `ERRORE FATALE: Sensori non calibrati` se omessa la fase K).

- **Comando `S` (SAFE - Messa in Sicurezza):** Commuta il flag `razzo_armato_per_volo = false`. Interrompe logicamente il collegamento tra l'output del miscelatore e le istruzioni hardware PWM. Imita l'inserimento della spina di sicurezza fisica (*Remove Before Flight pin*).
  Risultato a monitor: `[GCS] Sistema DISARMATO (Safe). Lancio bloccato.`

2. Calibrazione Algoritmica (Live PID Tuning)

Comandi utilizzati per la sintonia fine della dinamica di controllo. Il parser attende un valore a virgola mobile immediatamente successivo alla lettera.

- **Comando `P<valore>`** (es. `P2.5`): Sovrascrive il Guadagno Proporzionale (K_p). Definisce la reattività immediata del vettore alle deviazioni angolari.
  Risultato a monitor: `[LIVE TUNING] Nuovo Kp impostato a: 2.50`

- **Comando `I<valore>`** (es. `I0.15`): Sovrascrive il Guadagno Integrale (K_i). Compensa le deviazioni a lungo termine (es. asimmetrie costanti della spinta).
  Risultato a monitor: `[LIVE TUNING] Nuovo Ki impostato a: 0.15`

- **Comando `D<valore>`** (es. `D0.8`): Sovrascrive il Guadagno Derivativo (K_d). Agisce come smorzatore aerodinamico per prevenire sovracompensazioni (*overshoot*) e oscillazioni.
  Risultato a monitor: `[LIVE TUNING] Nuovo Kd impostato a: 0.80`

3. Calibrazione Cinematica (Servo Limits Tuning)

Comandi utilizzati per definire i vincoli meccanici degli attuatori, prevenendo lo stallo aerodinamico e il *binding* meccanico. Richiedono un valore intero (gradi).

- **Comando `C<valore>`** (es. `C90`): Imposta il centro nominale aerodinamico (Trim neutrale).
  Risultato a monitor: `[LIVE TUNING] Nuovo CENTRO servo impostato a: 90`

- **Comando `M<valore>`** (es. `M45`): Imposta l'escursione angolare minima consentita (Saturazione inferiore).
  Risultato a monitor: `[LIVE TUNING] Nuovo LIMITE_SERVO_MIN impostato a: 45`

- **Comando `N<valore>`** (es. `N135`): Imposta l'escursione angolare massima consentita (Saturazione superiore).
  Risultato a monitor: `[LIVE TUNING] Nuovo LIMITE_SERVO_MAX impostato a: 135`

- **Comando `O<valore>`** (es. `O0`): Calibra l'angolo di riposo del servomotore di recupero, garantendo che lo sportello o la molla del paracadute rimangano saldamente bloccati durante l'ascesa a Mach elevati.
  Risultato a monitor: `[LIVE TUNING] Nuovo ANGOLO_PARACADUTE_CHIUSO impostato a: 0`

4. Diagnostica e Telemetria su Richiesta (Data Polling & PFBIT)

Comandi di interrogazione e test da eseguire tassativamente prima della fase di armamento.

- **Comando `V` (View Constants):** Esegue un *dump* a monitor di tutte le costanti di volo attualmente caricate in RAM (PID e limiti). Essenziale per la verifica dell'operatore prima del decollo.
  Risultato a monitor: Stampa un prospetto riassuntivo con Kp, Ki, Kd, Centro e Limiti.

- **Comando `B` (Baro/Kalman Check):** Interroga l'osservatore di stato di Kalman per verificare che i filtri siano stabili. Il valore di velocità sulla rampa deve oscillare

strettamente attorno a 0.00 m/s.
Risultato a monitor: `[GCS] Dati Sensori: Altitudine da terra: 0.05 m | Velocità stimata: 0.01 m/s`

- **Comando `T` (Wiggle Test / Control Surface Check):** Esegue un Pre-Flight Built-In Test (PFBIT). Forza i quattro attuatori aerodinamici al loro limite massimo (`LIMITE_SERVO_MAX`) per 500 ms per poi riportarli al `CENTRO`. Permette un'ispezione visiva della fluidità dei leveraggi.
  Nota di Sicurezza: Il comando è via hardware interbloccato. Se eseguito con il vettore armato, la richiesta viene respinta per evitare sbilanciamenti catastrofici in caso di attivazione accidentale in volo.
  Risultato a monitor: `[GCS] TEST SERVI: Movimento in corso... Test servi completato. OK.` (Oppure messaggio di errore se armato).

- **Comando `X` (Parachute Deployment Test):** Attiva a secco il meccanismo di espulsione primario (`ANGOLO_SERVO_PARACADUTE_APERTO`) per un secondo, per poi richiuderlo. Consente di verificare la corretta tensione della molla di rilascio. Analogamente al Wiggle Test, è inibito in stato ARMED.
  Risultato a monitor: `[GCS] TEST PARACADUTE... Paracadute testato e richiuso.`

6.7 Passo 6 — Telemetria via LoRa

La funzione `inviaTelemetria()` assembla a ogni ciclo un **pacchetto CSV** con oltre 15 dati critici:

```
Tempo, Quota, Velocita, Apogeo_Flag,
Pitch, Roll, Yaw,
Accel_X, Accel_Y, Accel_Z,
PID_Pitch_Out, PID_Roll_Out, PID_Yaw_Out,
Servo_SX, Servo_DX, Servo_Avanti, Servo_Dietro,
Servo_Paracadute
```

Listing 6.1: Formato pacchetto CSV trasmesso a 5 Hz

Per non intasare la banda radio, il pacchetto viene trasmesso a intervalli regolari di 200 ms (5 Hz).

6.8 Passo 7 — Logica dell'Apogeo e Espulsione Paracadute

Il software traccia in continuo l'`altitudine_massima`. L'apogeo viene dichiarato quando si verificano **simultaneamente** tre condizioni:

1. Il razzo ha superato la **quota di sicurezza** di 15 m dal suolo;
2. La **velocità verticale** diventa negativa:

$$\texttt{stima_velocita} < -0{,}5 \text{ m/s}$$

3. L'**altitudine scende** di almeno 1,5 m rispetto al picco massimo registrato.

Azioni all'Apogeo

1. Il **servo del paracadute** scatta a APERTO (90°) — liberando la calotta di recupero;
2. Le **4 alette** vengono ruotate a `LIMITE_SERVO_MAX` per fungere da **aerofreno**, rallentando la discesa e stabilizzando il razzo mentre il paracadute si dispiega.

Se l'apogeo non è ancora stato rilevato, il codice esegue normalmente il PID e muove le alette per continuare l'ascesa verticale.

Capitolo 7

Miscelatore Aerodinamico e Attuazione Cinematica

7.1 Panoramica: Algoritmo di Control Allocation

La subroutine `mixer_servi` rappresenta lo stadio terminale della catena GNC (Guidance, Navigation, and Control). Il suo compito ingegneristico, noto come **Control Allocation**, è quello di tradurre i momenti torcenti desiderati — ovvero i valori astratti di correzione calcolati dai tre anelli PID indipendenti (Pitch, Roll, Yaw) — in deflessioni angolari fisiche per i quattro attuatori meccanici.

Poiché il razzo dispone di quattro superfici di controllo per governare tre gradi di libertà rotazionali, il sistema è matematicamente *sovra-attuato*. Ci troviamo di fronte a una cinematica complessa in cui gli effetti aerodinamici devono essere sommati algebricamente (**Principio di Sovrapposizione degli Effetti**) senza che le forze risultanti si annullino a vicenda.

7.2 Disaccoppiamento degli Assi e Logica Differenziale

L'algoritmo distribuisce il carico di lavoro sulle alette secondo una **geometria cruciforme**, dividendo le superfici in due coppie differenziali indipendenti, ma sovrapponendo a tutte un comando di autorità collettiva per il controllo assiale.

7.2.1 Autorità di Beccheggio (Pitch) e Rollio

Gestita primariamente dalla coppia longitudinale **Avanti e Dietro**. Per indurre un momento torcente puro attorno al baricentro (CG) e correggere l'assetto verticale, le due alette si deflettono in direzioni opposte rispetto al trim neutro. A questo differenziale

viene sommata algebricamente la componente di rollio:

$$\theta_{\text{Avanti}} = \text{CENTRO} + \text{out_Pitch} + \text{out_Roll}$$

$$\theta_{\text{Dietro}} = \text{CENTRO} - \text{out_Pitch} - \text{out_Roll}$$

7.2.2 Autorità di Imbardata (Yaw) e Rollio

Gestita in modo speculare dalla coppia trasversale **SX e DX**. Anche in questo caso, il comando differenziale di imbardata si fonde con il comando collettivo di rollio:

$$\theta_{\text{SX}} = \text{CENTRO} + \text{out_Yaw} + \text{out_Roll}$$

$$\theta_{\text{DX}} = \text{CENTRO} - \text{out_Yaw} - \text{out_Roll}$$

7.2.3 Autorità di Rollio (Controllo Collettivo)

A differenza del Pitch e dello Yaw, la stabilizzazione della rotazione assiale richiede che **tutte e quattro le alette deflettano all'unisono** nello stesso verso rotatorio. L'addizione della variabile `out_Roll` a tutte le equazioni genera un momento torcente assiale puro (effetto turbina), contrastando lo spin indotto dalle asimmetrie costruttive delle alette senza alterare la traiettoria verticale del vettore.

7.3 Gestione della Saturazione e Clamping Elettronico

Sommatura Costruttiva e Rischio di Saturazione

Un rischio intrinseco e letale nei sistemi di *Control Allocation lineare* è la sommatura vettoriale costruttiva delle variabili. Qualora il vettore subisca contemporaneamente una severa raffica di vento al traverso (elevata richiesta di `out_Yaw`) e un'anomalia rotazionale (elevata richiesta di `out_Roll`), la sovrapposizione algebrica dei due comandi PID potrebbe generare un *setpoint* angolare esterno ai limiti fisici dell'attuatore (es. un comando teorico di 170°).

7.3.1 Modalità di Guasto per Saturazione Non Gestita

L'assenza di un filtro di contenimento angolare porterebbe a tre scenari di cedimento catastrofico (Catastrophic Failure):

1. **Stallo Meccanico (Mechanical Binding):** La squadretta del servomotore o i leveraggi interni collidono fisicamente contro la fusoliera o i finecorsa strutturali, bloccando l'attuatore.
2. **Collasso Elettrico (Brownout):** Un servomotore in stallo meccanico assorbe correnti estreme (Stall Current). Questo drenaggio istantaneo può causare il crollo della tensione del bus di alimentazione (Voltage Drop), innescando il reset hardware del microcontrollore e spegnendo il computer di volo a mezz'aria.
3. **Stallo Aerodinamico (Fin Stall):** Fluidodinamicamente, un'aletta flessa oltre i **15°-20°** rispetto al flusso d'aria laminare subisce il distacco dello strato limite (Boundary Layer Separation). La superficie perde drasticamente portanza (Lift) e innesca una massiccia generazione di resistenza parassita (Drag), frenando violentemente il razzo e rendendo i comandi del PID totalmente inefficaci.

7.3.2 La Soluzione Matematica: Electronic End-Point Adjustment (EPA)

Per garantire l'integrità del vettore, il firmware implementa la funzione di saturazione `constrain()`. Essa agisce come un **Electronic End-Point Adjustment (EPA)** di sicurezza ferrea. Indipendentemente dalla gravità dell'errore calcolato dai PID, l'output angolare (θ_{raw}) per ogni singolo attuatore viene **clampato** matematicamente entro una "finestra" di sicurezza predefinita:

$$\theta_{\text{cmd}} = \max(\theta_{\text{min}}, \min(\theta_{\text{raw}}, \theta_{\text{max}}))$$

Nel codice C++, questo si traduce in:

$$\theta_{\text{cmd}} = \text{constrain}(\theta_{\text{raw}}, \text{LIMITE_MIN}, \text{LIMITE_MAX})$$

Grazie a questa barriera software, le superfici di controllo non supereranno mai i **45°** e i **135°** (ovvero una deflessione aerodinamica massima di $\pm 45°$ dal centro), proteggendo l'integrità strutturale, elettrica e fluidodinamica del sistema in ogni regime di volo.

Appendice A

Glossario dei Termini Tecnici

IMU *Inertial Measurement Unit* — unità di misura inerziale che combina accelerometro, giroscopio e magnetometro.

PID *Proportional-Integral-Derivative* — algoritmo di controllo a retroazione che minimizza l'errore tra il valore desiderato e quello misurato.

DCM *Direction Cosine Matrix* — matrice di coseni direttori usata per la rotazione di vettori tra sistemi di riferimento.

EMA *Exponential Moving Average* — filtro digitale IIR del primo ordine per la riduzione del rumore.

LoRa *Long Range* — tecnologia radio a bassa potenza e lunga portata per la trasmissione dati.

GCS *Ground Control Station* — stazione di controllo a terra.

GNC *Guidance, Navigation and Control* — sistema integrato di guida, navigazione e controllo.

PWM *Pulse Width Modulation* — tecnica di modulazione usata per controllare i servomotori.

Pitch Beccheggio — rotazione attorno all'asse trasversale (avanti/indietro).

Roll Rollio — rotazione attorno all'asse longitudinale (spin assiale).

Yaw Imbardata — rotazione attorno all'asse verticale (destra/sinistra).

Body Frame Sistema di riferimento solidale al vettore (razzo).

Inertial Frame Sistema di riferimento fisso rispetto al suolo.

Apogeo Punto più alto della traiettoria balistica, dove la velocità verticale è zero.

Windup Effetto di saturazione dell'integratore PID che causa movimenti violenti e incontrollati.

Binding Stallo meccanico di un servomotore causato dal superamento dei limiti fisici.

Brownout Crollo della tensione di alimentazione causato da un assorbimento eccessivo di corrente.

PFBIT *Pre-Flight Built-In Test* — diagnostica integrata eseguita prima del volo.

FSM *Finite State Machine* — macchina a stati finiti che governa la logica di sicurezza.